EECS1516 Assignment 2: KD-Trees

1 Introduction

Images in this specification are from Carl Kingsford's lectures on KD-Trees

You have now seen several ADTs in Python that manage collections of objects, like stacks, trees and queues. In your first assigment, you built a simple tool to ask the question "what value in this collection is most similar to item X?". In this assignment, you will build an abstract data structure called a **kd-tree** (short for "k-dimensional tree") that can be used to answer the same kind of question. You will use properties of this structure in order to find items 'nearest' a data point and you will then use information to predict properties of the data point. Once again, we will be using RentSafeTO data to fuel our inquiry; more specifically, we will predict ownership of buildings based on a selection of inspection assessments.

2 KD-Trees: A primer

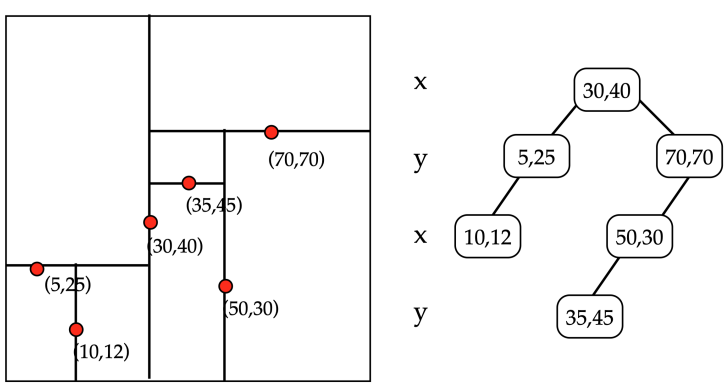

Figure 1. A KD-Tree where $K = 2$. At left, an illustration of the way the tree partitions 2-dimensional space. At right, the nodes in the KD-tree. Note that in this homework, you will be partitioning 3-dimensional space!

A kd-tree is a form of a binary search tree that stores points in k-dimensional space. You can use a kd-tree to store a collection of points on a plane or in higher-dimensional space. In this assignment, you'll be building a kd-tree to store data about apartment buildings and to quickly find buildings based on the assessments of inspectors on a number of items.

At the root of a kd-tree, the **x-value** in the coordinate pair is used to separate nodes at the next level. All of the nodes to the right of the root have a larger or equal x-value than the node at the root and all nodes to the left of the root have a smaller x-value than the node at the root.

At the next level in the kd-tree it is the **y-value** that is used to separate nodes. In Figure 1, the nodes to the left of the one labelled (70,70) all have a y-value less than 70. The nodes to the right (if there were any) would all have a y-value greater than or equal to 70.

Because of the way data is organized within this kd-tree, we can efficiently query whether a given point exists within it as follows. Given a coordinate pair (X,Y), we start at the root of the kd-tree. If the root node contains (X,Y), we know the value exists in the kd-tree. However, if the x-value is strictly less than the x-value of the root node, we will search for (X,Y) in the left subtree. Otherwise, we will descend into the right subtree. When we descend a level, we will again ask if the node we are exploring contains (X,Y). If not, we will compare the y-value in our coordinate pair to the y-value at this node. If the y-value in the coordinate pair is strictly less than the y-value of the node, we will search for (X,Y) in the left subtree and otherwise, we will descend into the right subtree. We will continue this process, alternating between x and y comparisons at each level, until we reach the leaves of the kd-tree. We will either find the value we seek there or fail, in which case we know the value does not exist in our collection.

Your assignment will be to a KDTree class in Python that allows clients to build kd-trees and perform lookup operations with them. While the examples above relate to kd-trees in 2-dimensional space, the trees you write here will partition a three dimensional space defined by three assessments from building inspectors. We will specifically encode operations to assess if a value exists in the tree as well as operations to locate points that are similar or "near" a given query. We will then use the properties of the tree to locate points that are "near" a given query point in order to predict properties of our query. We will describe each operation in more detail below.

3 Step 1: Tree Construction

Constructing a KD-tree involves recursively partitioning the space in which our data points lie into smaller regions. The algorithm below assumes our data contains points in a 2-dimensional space and that the root of our KD-tree is an-

chored at a **depth of 0**. When you build your own KD-tree, your algorithm will be mindful of THREE dimensions that are associated with apartment building objects: the cosmetic, moderate and high risk assessment scores. Apartment building objects will be wrapped within nodes of the tree (called KDNodes) and associated with other attributes, like a location, ownership information, year of construction and overall evaluation 'score'. However, we will not need to consider the values of these other attributes during tree building process.

Our kd-tree construction process (which you will encode in the $build_tree(apartment_data, depth)$ method) follows; in it, the first coordinate (c) is defined by an apartment building's cosmetic assessment score, the second (m) its moderate risk assessment score and the third (h) its high risk assessment score:

- If there is only one data point (i.e. one apartment building) in your data set, we will create a single leaf (or KDNode) to represent this point and return it. This will be the root of our tree, and we're done.
- Otherwise and if **depth mod three is 0**, we will split the data into two subsets based on the value of the **cosmetic assessment score, c**. Call P1 the set of points with cosmetic scores strictly less than the **median value of c**, and let P2 be all the remaining points.
- If instead the **depth mod three is 1**, we will split the list of input points into two subsets based on the value of the **moderate risk assessment score, m**. Call P1 the set of points with moderate risk assessments strictly less than the **median value of m**, and let P2 be all the remaining points.
- Finally **depth mod three is 2**, we will split the list of input points into two subsets based on the value of the **high risk assessment score, h**. Call P1 the set of points with high risk assessments strictly less than the **median value of h**, and let P2 be all the remaining points.
- We will now recursively call 'build_tree' with input parameters P1 and depth+1 (i.e. $build_tree(P1, depth + 1)$), and we will assign the KDNode returned from this recursive call to the variable ($vleft$).
- We will also recursively 'build_tree' with input parameters P2 and depth+1 (i.e. call $build_tree(P2, depth + 1)$), and will assign the KDNode returned from this recursive call to a variable ($vright$).
- Finally, we will manufacture a KDNode. We will make $vright$ the right child of this node and $vleft$ the left. We will also make sure that our KDNode stores the data point associated with the median (i.e. with the point that separates those in $vleft$ from those in $vright$).
- When we are done, we will return the KDNode we have constructed. Note that this means a complete KD-tree can be defined by a single attribute: the KDNode that lies at the **root**.

To complete our kd-tree initialization, we will also encode another method named *calculate_year_average*. This method will traverse the entire tree returned by the *build_tree* method and accumulate the values of every apartment construction *year* contained within the tree. It should then divide this accumulated value by the number of nodes contained within the tree in order to produce an 'average' construction year. Store this average as an integer and as an attribute of the KD-tree.

4 Step 2: Point Lookup

Within a KD-tree, we can search for a specific point recursively. The steps described below assume, once again, that we are searching in a 2-dimensional space. More specifically, the first coordinate (c) is assumed to be a cosmetic score associated with an apartment building, the second (m) is its moderate risk score and the third (h) its high risk assessment score. The lookup process (which you will encode in the $lookup_apartment(c, m, h)$ method) follows:

- If you encounter a KDNode that is a leaf, assess if the values of the apartment contained there equals your query. If yes, your query is represented in the tree. If no, your query is not represented in the tree. Your search is complete at this point.
- Otherwise and if you are at a KDNode where **depth mod 3 is 0**, compare the **cosmetic score, c** of your query point to the cosmetic score of the apartment at that KDNode. If the value of the query is strictly less than the KDNode's cosmetic score, recursively search the **left** subtree of the given KDNode. Otherwise, search the **right** subtree.
- If instead you are at a KDNode where**depth mod 3 is 1**, compare the **moderate risk score, m** of your query point to the moderate risk score of the apartment at that KDNode. If the value of the query is strictly less than the KDNode's moderate risk score, recursively search the **left** subtree of the given KDNode. Otherwise, search the **right** subtree.
- Finally, if you are at a KDNode where **depth mod 3 is 2**, compare the **high risk score, h** of your query point to the high risk score of the apartment at that KDNode. If the value of the query is strictly less than the KDNode's high risk score, recursively search the **left** subtree of the given KDNode. Otherwise, search the **right** subtree.

5 Step 3: Finding the Closest Point to a Query

A closest-point search will ask, given a set of coordinates (c,m,h), to locate the point P representing an apartment that is **closest** to (c,m,h). It would be great if we could use our *lookup_apartment* to help us here, but there's a catch. This is because the data in any leaf we encounter when we search for a specific

point may not actually be all that close to the query! For example, consider what would happen if we searched for the point (52,52) in the two dimensional KD-tree below:

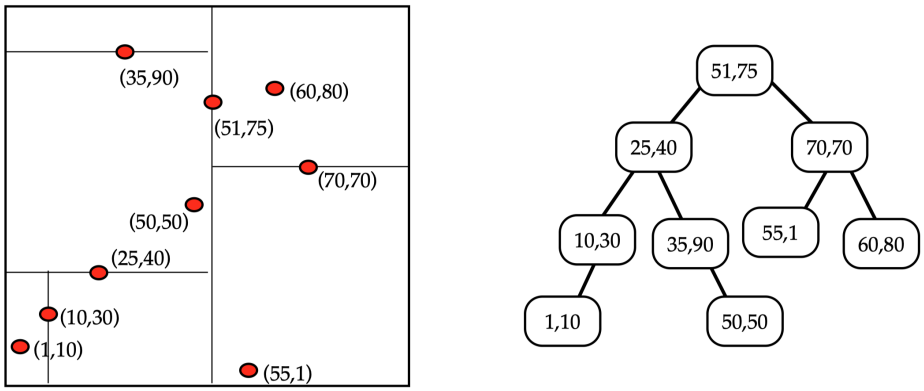

Figure 2

A naive search for (52,52) will lead us to the leaf containing the value (55,1). But this is quite distant from our query of (52,52); the point (51,75) is actually much closer.

To address the issue here, we must modify our *lookup_apartment* operation in order to implement a new method we will call *get_closest_point*. Our implementation will still be recursive, but we will keep a record of the closest point we have discovered so far and the distance to this closest point while we are searching. The algorithm will proceed as follows:

- Before we start, we will initialize the closest point found so far to $None$ and the closest distance found so far to $float('inf')$ (i.e. a representation of infinity).
- As we are searching, if we hit the **bottom** of our KDTree, we should **return** the closest point and closest distance found thus far.
- If we are at a KDNode en route to the bottom, we should instead compare the value stored in this KDNode with our query. If the value is **closer** than the closest point found so far, we should replace the record of the closest point found so far with this value. We should also update the closest distance accordingly.
- Next, recursively search the subtree with the **greatest promise** of containing a point closer than the closest point found so far. More specifically, if we are at a depth of the KD-tree where **depth mod 3 is 0** and the **cosmetic score** in our query is strictly less than that in the current node, consider the **left** subtree to be the **most promising** (and if not, we will

focus on the **right** subtree). If instead **depth mod three is 1** and the **moderate risk score** of our query is strictly less than that of the current node, we will call the **left** subtree the **most promising**. And, if **depth mod three is 2** and the **high risk score** of our query is strictly less than that of the current node, call the **left** subtree the **most promising** and continue your recursively search from there. Once we have identified the **most promising** subtree, we will recursively search it and save the closest point and closest distance returned by this recursive call.

- Finally and importantly, we must decide if it is worth our effort to search the **less promising** subtree. We will make this decision by calculating the distance between our current closest point and the value used to partition the left and right subtrees at the current node. At levels where **depth mod three is 0**, the partitioning value will be a **cosmetic score**; if **depth mod three is 1** it will be the **moderate risk score** and if **depth mod three is 1** it will be the **high risk score**. Either way, if the distance between the partitioning value and the corresponding coordinate in our query is **less than** the distance between the query and the closest point found thus far, we must recursively search the **less promising** subtree. But if not, we can safely ignore this subtree entirely.

- Once all our recursive searches are complete, we can return the closest point and closest distance we have identified across all of our recursive calls. And then we are done!

6 Step 4: Classification

In the last assignment you encoded a simple **nearest neighbour classifier**, and you will do the same here. This time, however, your data structure will support an efficient search! More specifically, you are asked to encode a **nearest neighbour classifier** that predicts the owners of apartments based on the properties of nearby neighbors stored in your kd-tree. To do this, you will write the method *predict*, which will accept a list of previously unseen apartment buildings and return a list of predictions for the building owners ('SOCIAL HOUSING', 'PRIVATE' or 'TCHC') that is equal in length. The algorithm you will use follows:

- For each apartment building (Y) in the input list, locate the apartment nearest that building in your kd-tree. Call the nearest neighbour X.

- Predict that the property owner of X is the **same** as the property owner of Y. For example, if 'TCHC' owns X, assume it owns Y as well. Store your classification in a list of strings.

- Once you have classified every apartment in the input list, return the list of string classifications.

Note that when we make classifications, we will be using data about apartments that are NOT represented in our KD-tree. We have included one methods to help you evaluate, in a rudimentary way, the quality of your classifications; this is called *get_ground_truth*. We ask that you implement *calculate_accuracy* to return a percentage of correctly guessed labels given the ground truth labels and a set of predictions. Make sure that you understand all of these methods and note that we will discuss more robust evaluation strategies in class.

7 What You Will Implement

You will then implement the following method (worth 10%) in $_KDTNode$ class:

- $distance(self, c : float, m : float, h : float) - > float$, which will calculate the 3-d distance between the assessments of the apartment that is wrapped within a given $_KDTNode$ and the location defined by the (c, m, h) coordinates. You can define this distance using the **Euclidean distance** formula as follows: $\sqrt{(c_2 - c_1)^2 + (m_2 - m_1)^2 + (h_2 - h_1)^2}$.

You can then implement the following methods in the $KDTree$ class:

- $arg_sort(seq : list[int])$. This is a static method returns the indices of values in a sequence such that the values are listed in ascending order. For example, if the input is $[8, 6, 7, 5, 3, 0, 9]$ the output should be $[5, 4, 3, 1, 2, 0, 6]$. This is because the fifth item in the input list has the lowest value, the fourth item has the second lowest value, and so on. Note that the median value of the input list will be in the centre of the output list. This method may be helpful to you when you partition input values into subsets based on median values during your construction of a $KDTree$. (worth 10%)

- The method $build_tree(self, data : list[Apartment], depth : int)$, which will build a KDTree that contains one node for each element in the input data that is provided. This will be a recursive method, and is described in Step 1. You will build the tree by determining the median of the data at each level of the tree and then partitioning the data around that median. Remember to use **depth mod 3** to determine which attribute you use for the purpose of partitioning. (worth 20%)

- The method $calculate_year_average(self)$, which will calculate the average 'year' of construction for apartment buildings contained within the $_KDTNode$s of a given tree. (worth 15%)

- The method $lookup(self, c : float, m : float, h : float)l$, which will return $True$ if there is any data point in the $KDTree$ with the given c, m and h coordinates. Otherwise, it will return $False$. Refer to Step 2 for more details on the implementation of this method. (worth 10%)

- The method $get_closest_point(self, c : float, m : float, h : float)$. This method will return the apartment building contained in the $KDTree$ that is nearest to the given c, m and h coordinates. Refer to Step 3 for more details on the implementation of this method. (worth 15%)

- The method $predict(self, test_data : list[Apartment])$. This method will classify each apartment building in the input list $test_data$ as being either 'SOCIAL HOUSING', 'PRIVATE' or 'TCHC'. Classifications will be stored in a list of strings that is returned. Refer to Step 4 for more details on the implementation of this method. (worth 10%)

Finally, implement:

- $calculate_accuracy(predictions : list[str], truth : list[str]) -> int$. This should accept a list of ground truth labels and a list of predicted labels. Output is an integer between 0 and 100 that represents the percent of predicted labels that are correct (worth 10%)

8 Deliverables

You will submit your modified `kdtree.py` using eClass. It is your responsibility to include all necessary files in your submission. You can submit a new version of any file at any time, though the lateness penalty applies if you submit after the deadline.

Important corrections (hopefully few or none) and clarifications to the assignment will be posted to eClass. Questions about the assignment can be asked on eClass. If you have a question of a personal nature, please email sallin at yorku.ca, placing [EECS1516] in the subject header.

A significant portion of your marks will depend on electronic unit tests. We will NOT be marking your assignment based on the autograder alone; this is provided to you simply to help you know that you're working in the right direction. **It's up to you to create test cases to test your code - that's part of the assignment!**

Have fun and GOOD LUCK!!